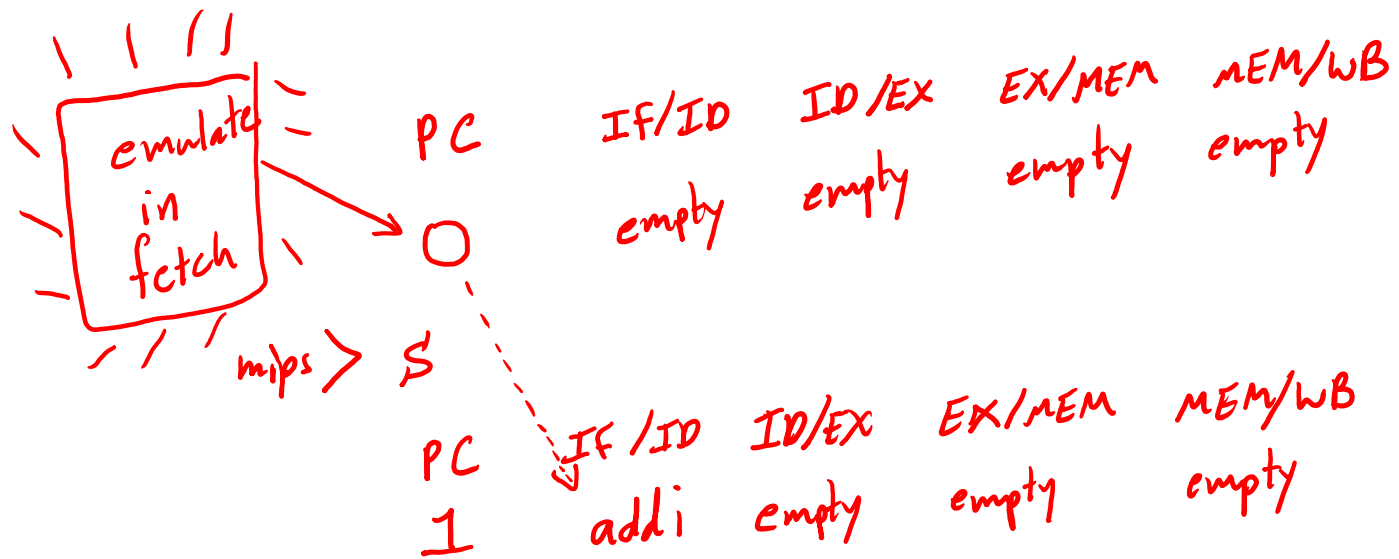


Lab 4 (Mon. 5/18)

Lab 3 - ^{CPU}emulator - runs inst.

Lab 4 - CPU simulator - outputs cycle counts



when program ends print the # of cycles,
of inst,
and CPI

- Lab 4

- Midterm Monday

PDF on Canvas { - You will get a copy of the MIPS ref. sheet

- 50 min. test on paper

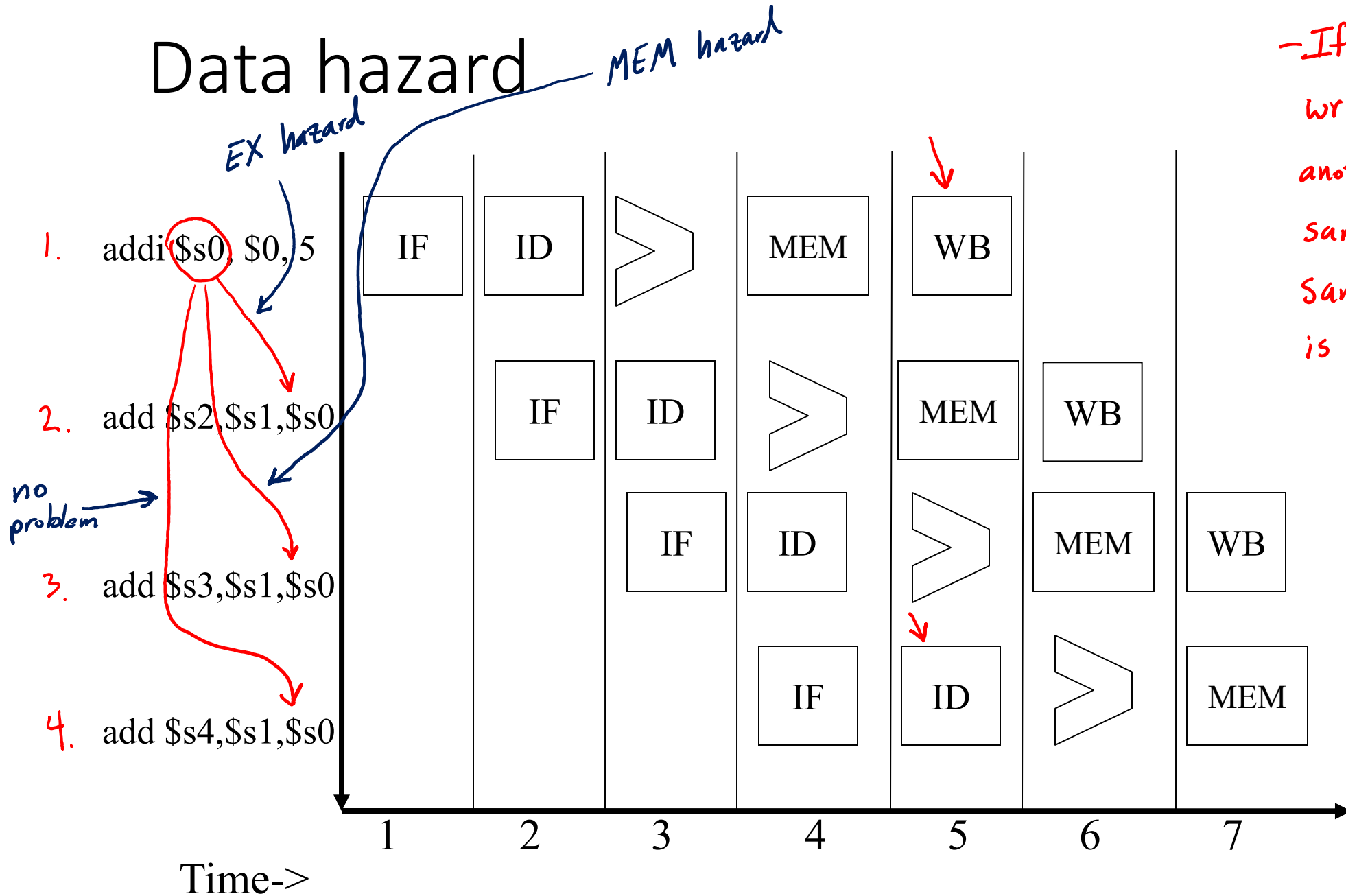
know how read op/funct, register names

PC
0 addi
1 add

~~Jan~~

j/and

Data hazard

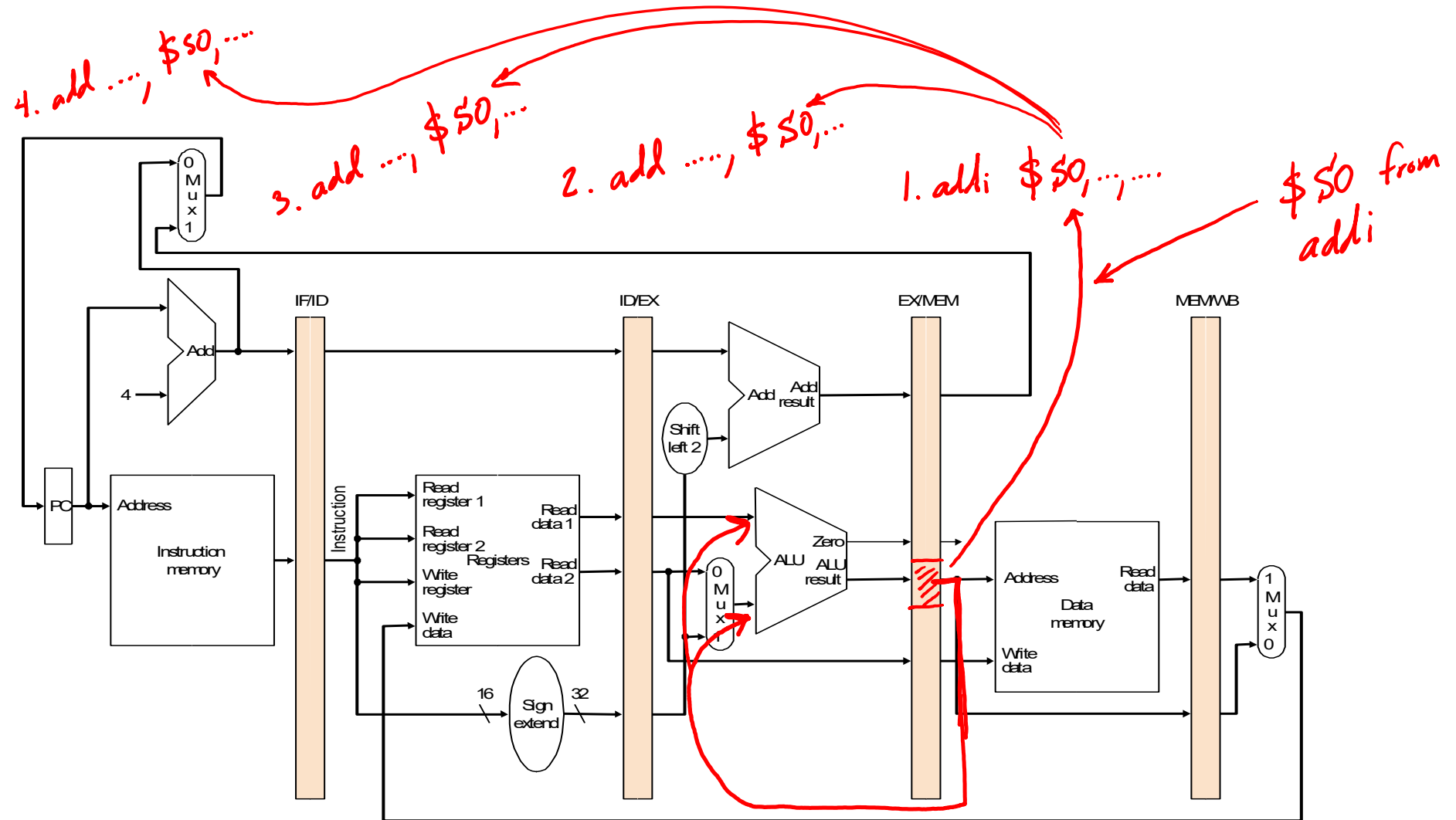


-If an inst. writes a reg., and another reads the same reg. in the same cycle, there is no problem

EX addi \$s0, ...
add ..., \$s0

MEM addi \$s0, ...
X
add ..., \$s0

Data hazard



Data hazards

- Needed data value: ^($\$S0$) the value is computed but not yet in the register file
 - Solution: Copy the value from the pipeline register where the value is currently stored
- 
- forwarding

How to detect a data hazard

EX hazard:

addi \$S0, ..., ...
add ..., \$S0, ...

If (EX/MEM. RegWrite == 1 and

EX/MEM. RegRD == ID/EX. RegRS and

special case { EX/MEM. RegRD != 0)
then Forward the Value

MEM hazard:

addi \$S0, ..., ...
X
add ..., \$S0, ...

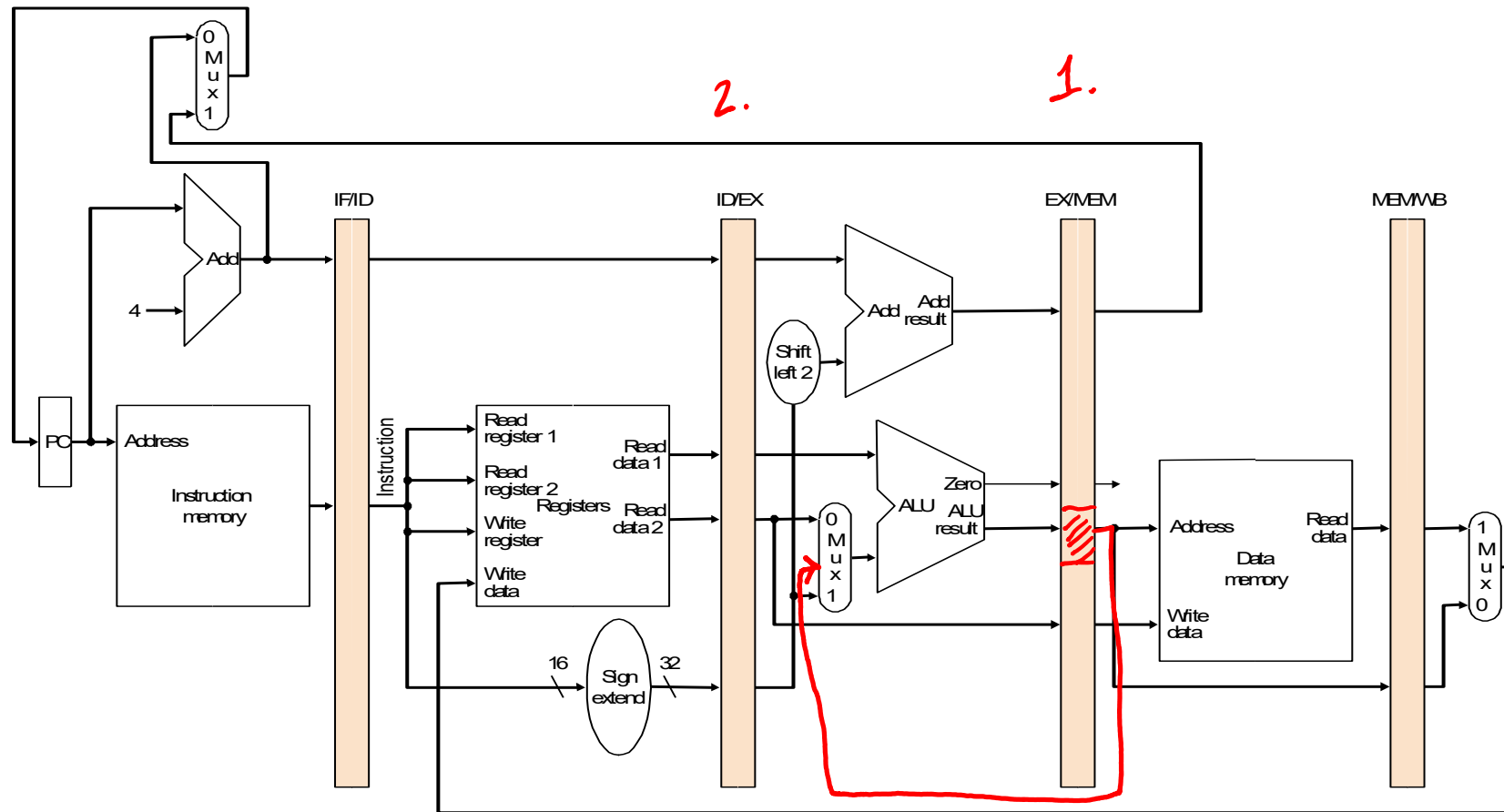
If (MEM/WB. RegWrite == 1 and

MEM/WB. RegRD == ID/EX. RegRS and

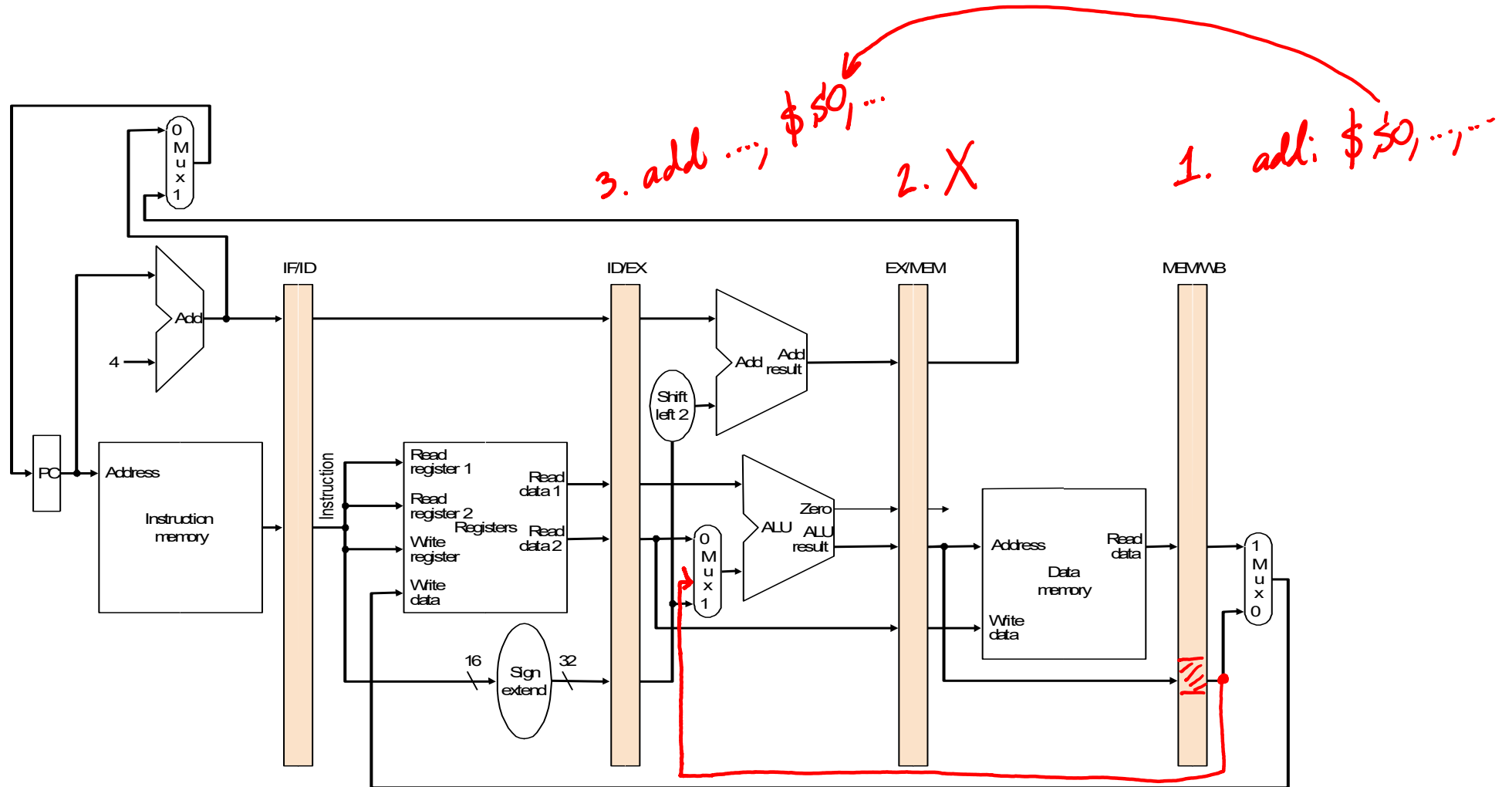
MEM/WB. RegRD != 0)

then Forward the Value

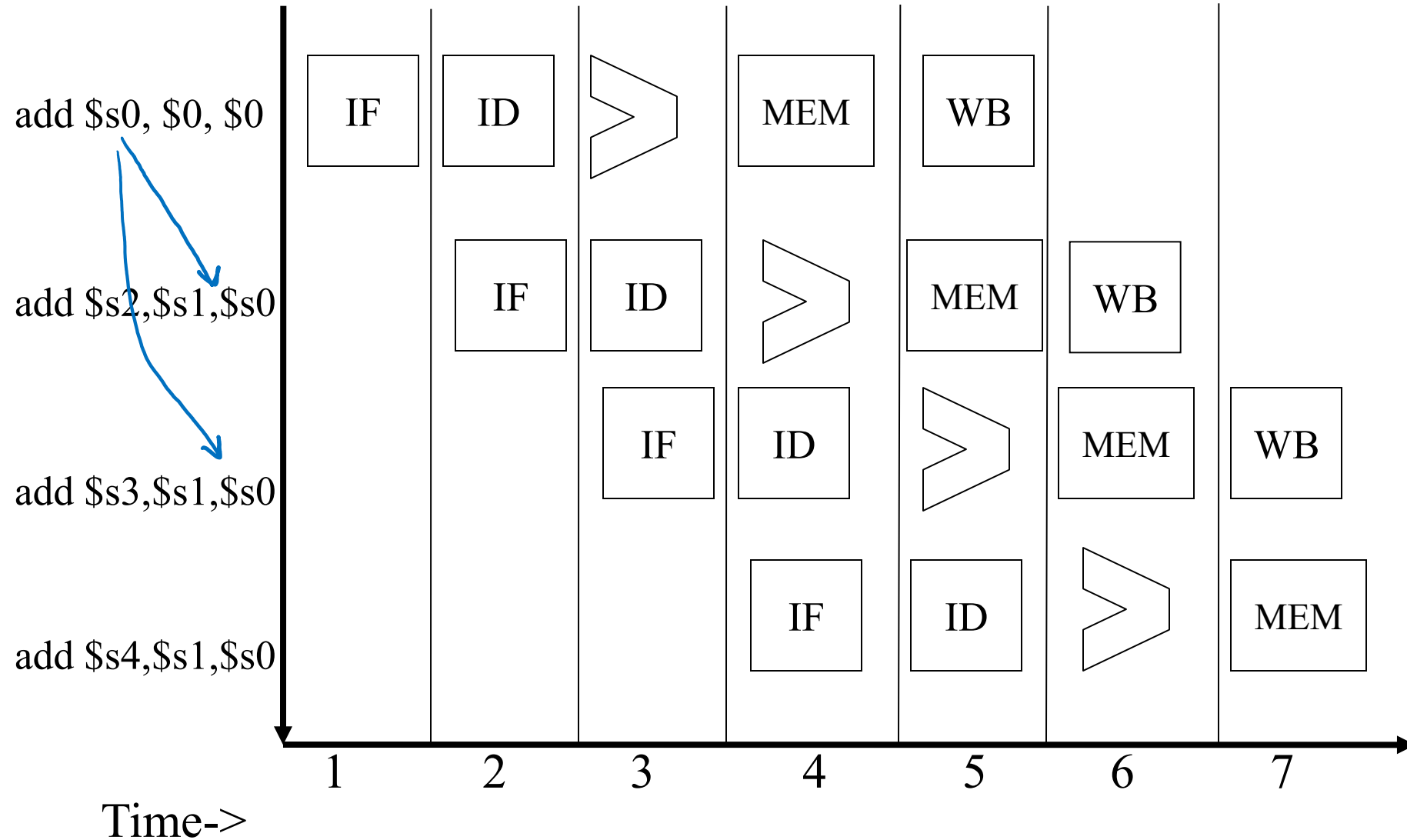
Forwarding for EX hazards

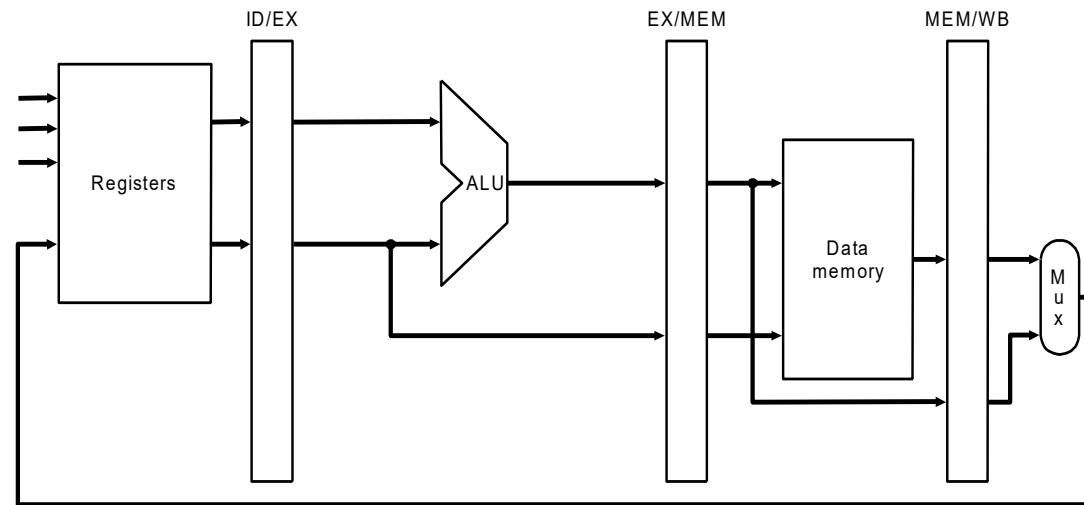


Forwarding for MEM hazards

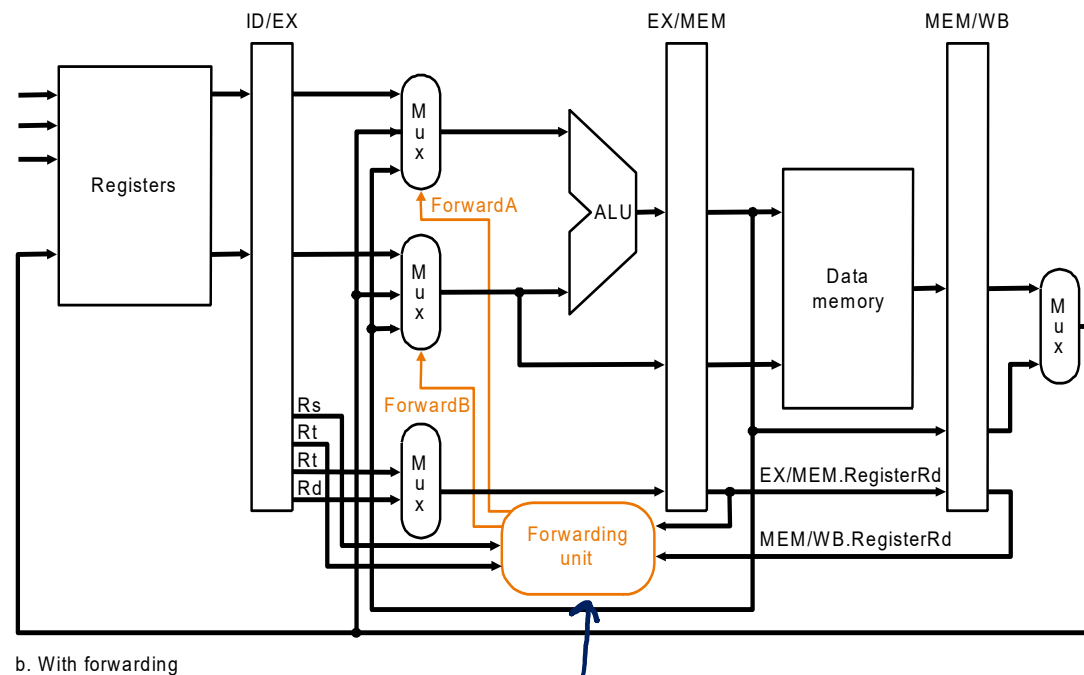


With Forwarding





a. No forwarding



b. With forwarding

Forwarding
Unit =
contains the logic
of the previous
"IF" expressions

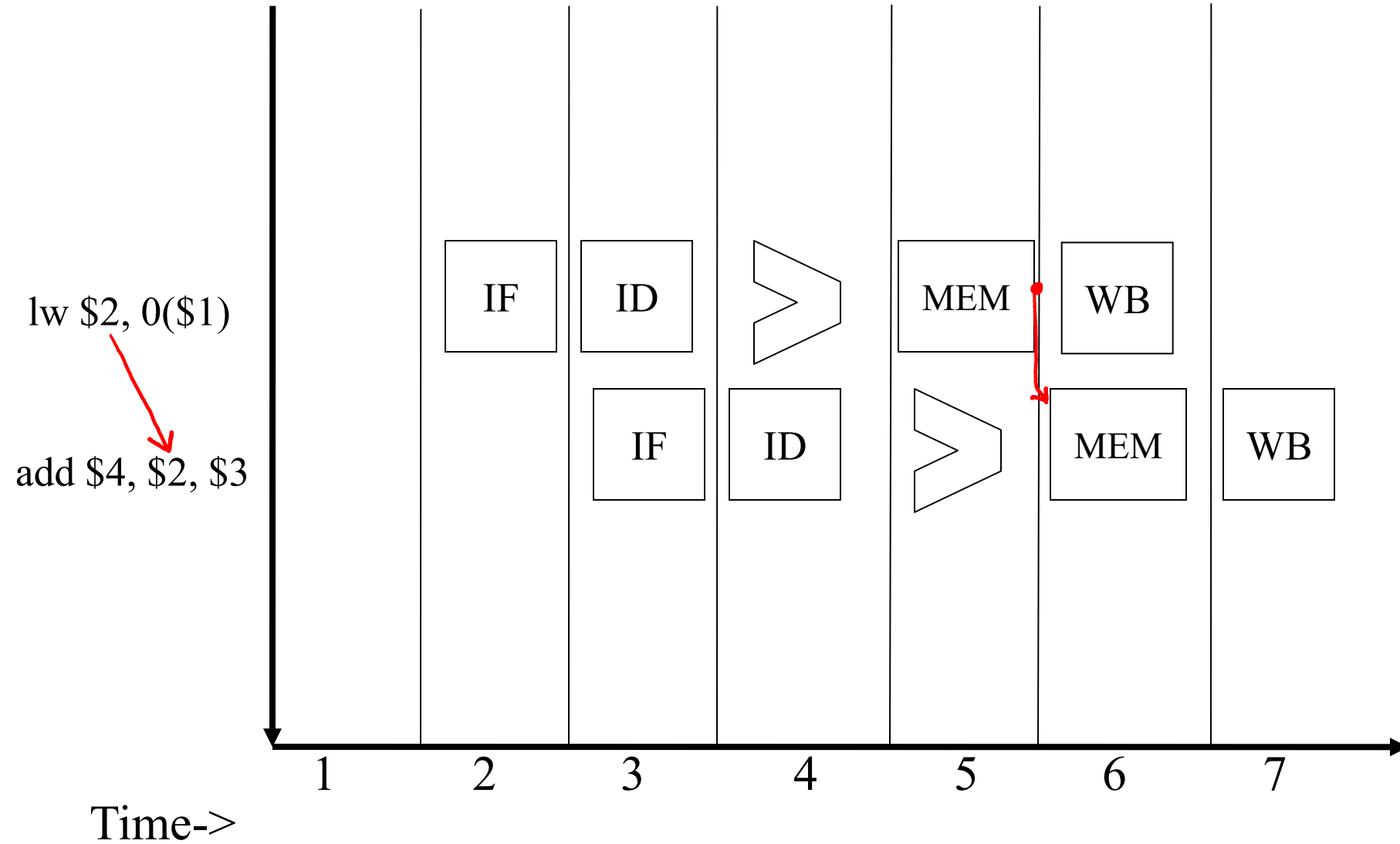
Forwarding limitations

- Forwarding fixes most problems
- Except one:

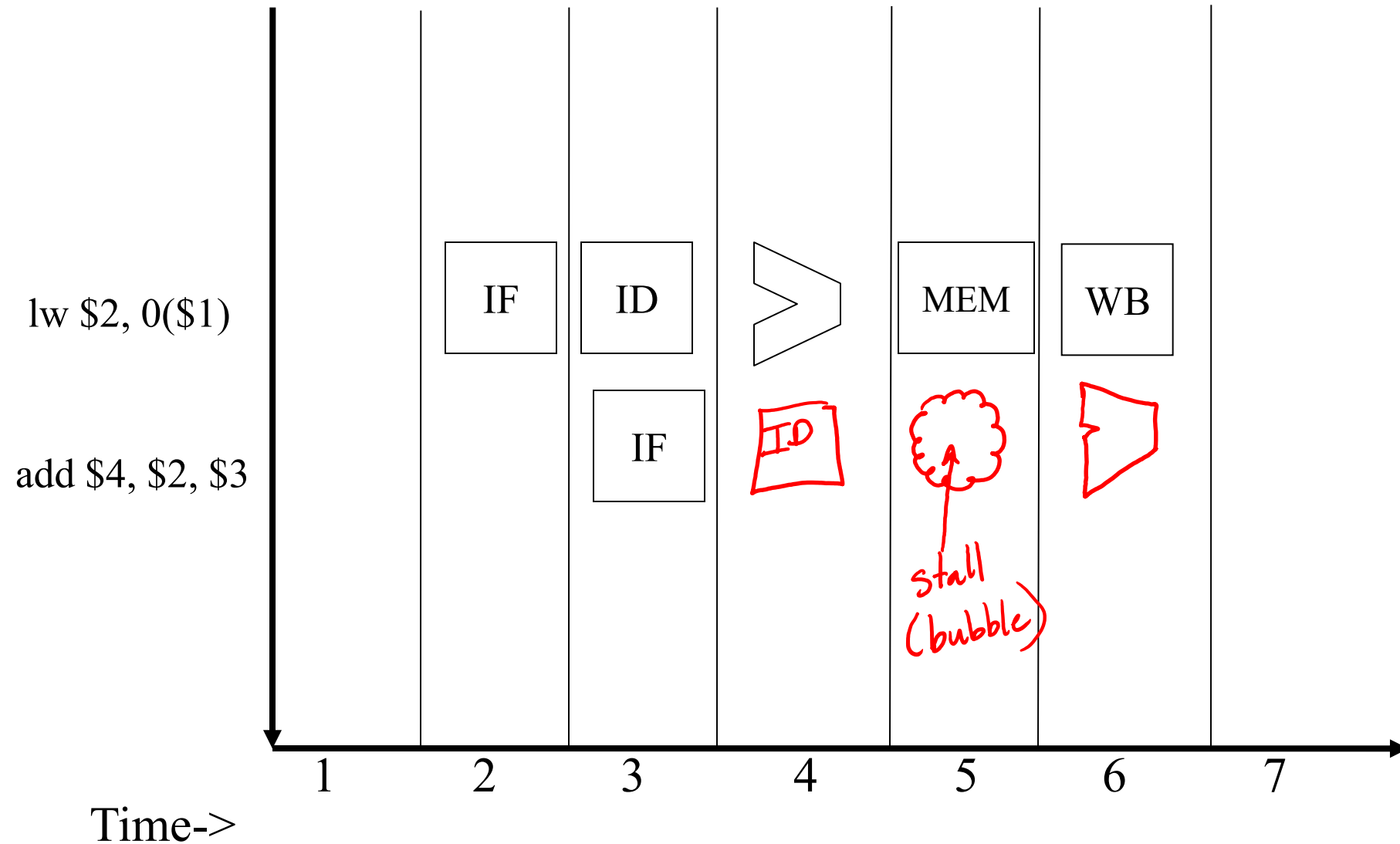
lw \$a0, 4(\$sp)
add \$t0, \$a0, \$s0

*} use after load
(use reg. immediately
after the load
inst.)*

Use after Load



Solution: Use after Load



How to detect a use-after-load

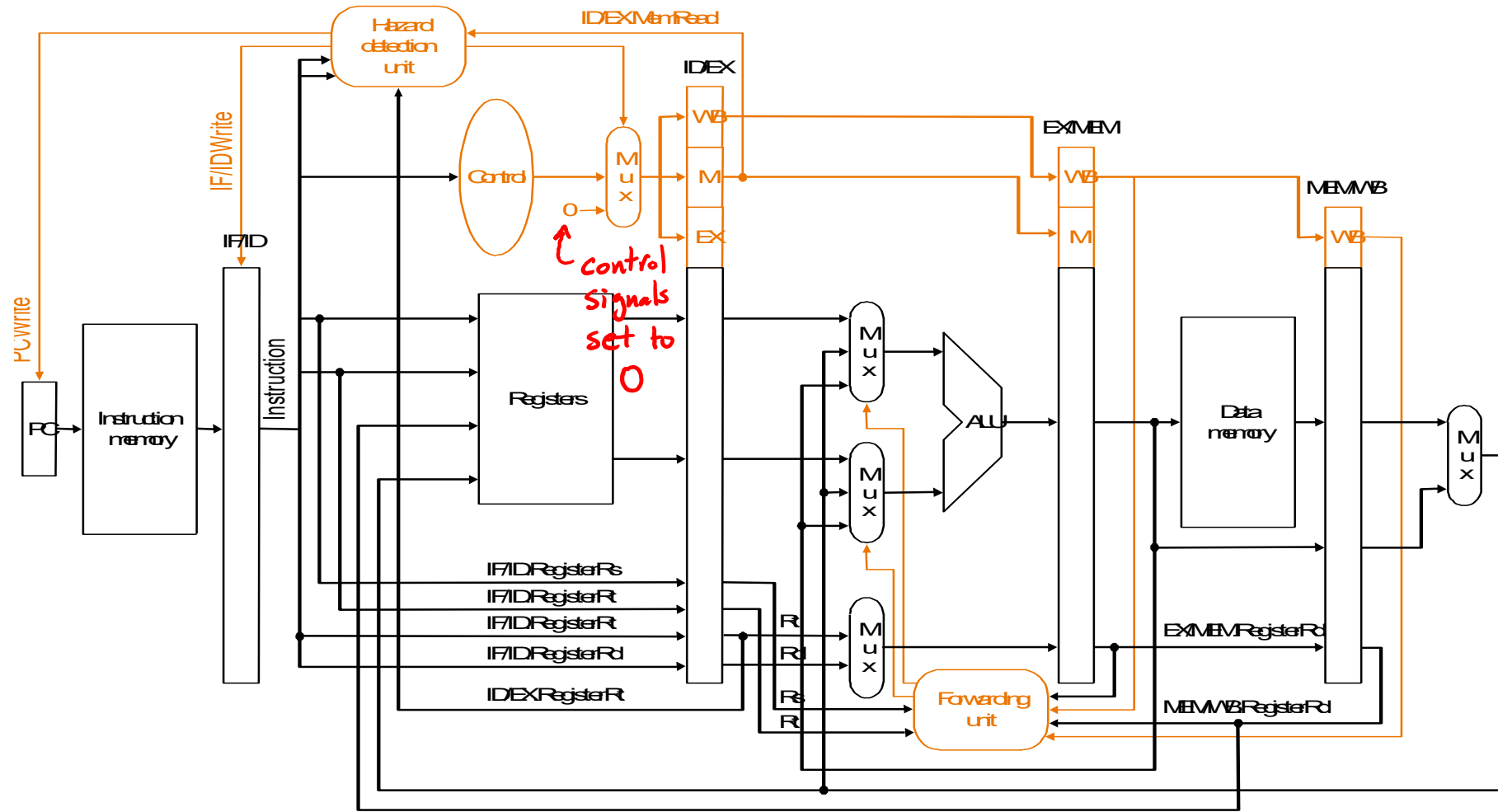
lw \$a0,
add ..., \$a0, ...



If (ID/EX. opcode == lw and
(ID/EX. RegRT = IF/ID. RegRS or
ID/EX. RegRT = IF/ID. RegRT) and
ID/EX. RegRT != 0)
then Stall the pipeline

On a stall:

- turn of PC write enable
- turn of IF/ID write enable
- set all control signals
in ID/EX to 0 (no-op)



Eliminating pipeline stalls

lw \$a0,
add, \$a0,
add \$t0, \$t1, \$t2



lw \$a0,
add \$t0, \$t1, \$t2
add, \$a0,



faster
(no stalls)